

InfraPilot — 4-Week Implementation Plan

What we're building

A natural-language DevOps assistant for AWS ECS on Fargate. Users type plain English like "deploy my app on port 3000" and the system interprets the request, generates Terraform code, and executes it after user confirmation. The backend lives in the cloud. The CLI is a thin executor on the user's machine.

What's in scope (6 operations)

Setup infrastructure, deploy a service, scale a service, stop a service, tear down a service, tear down all infrastructure. These cover the complete ECS lifecycle from first cluster to final teardown.

Architecture

The backend does the thinking: it understands what the user wants through LLM, decides what steps are needed, and generates the required Terraform code by filling out a template. The CLI does the doing: it runs Terraform, Docker, and AWS CLI commands locally where the user's AWS credentials and Docker daemon live.

The two communicate over two HTTP calls. The first sends the user's natural language input plus project context to the backend and receives a structured execution plan with generated Terraform files. The second sends Terraform outputs (resource IDs created during execution) back to the backend so it can update project state.

The backend stores two things in PostgreSQL: a users table that maps API keys to identities, and a project state table that tracks what infrastructure exists for each user's project (cluster ARN, VPC ID, ALB DNS, ECR URL, subnet IDs). This state is essential — it's how the NLP layer knows whether infrastructure exists and can make smart decisions like automatically prepending a setup step before a deploy.

Why these technologies

FastAPI because it's lightweight, auto-generates documentation, and handles async. The entire backend stays small.

OpenAI function calling because it returns structured JSON matching schemas you define. You describe your 6 intents as function definitions, the model returns the intent name and extracted entities in a predictable format. No output parsing needed.

PostgreSQL because your data mixes structured fields (user ID, project name) with variable-shape JSON (NLP entities differ per intent, Terraform outputs differ per resource). JSONB columns handle both. Managed hosting on Neon or Supabase means zero maintenance.

Terraform with Jinja2 templates because templates are tested once and produce identical results every time. The NLP layer extracts parameters, the templates fill them in. Deterministic and fast, unlike LLM-generated infrastructure code which varies per run.

ECS on Fargate because it's a single AWS-native plane. No Kubernetes layer, no kubeconfig, no node management. The entire concept space fits in 6 intents.

Cloud Run for hosting the backend because it auto-scales from zero, runs containers, and costs nearly nothing at low volume.

Python Click + Rich for the CLI because Click handles command structure and Rich handles formatted tables, progress spinners, and colored output.

Part 1 — Person A: Backend Core

Person A owns the FastAPI application, authentication, database, cloud deployment, and endpoint integration. They are the integration point that everyone else plugs into.

Week 1: Standalone backend

Stand up the FastAPI application with two endpoints. The interpret endpoint accepts user input and project context, validates the API key, and returns a response. The state endpoint accepts Terraform outputs and stores them. Both return mock data initially.

Set up PostgreSQL with two tables. The users table has an ID, API key string, name, and creation timestamp. The project state table has an ID, user reference, project name, region, an infrastructure state column (JSONB storing cluster ARN, VPC ID, ALB DNS, ECR URL, subnet IDs, security group IDs), a services column (JSONB tracking deployed services), and an update timestamp. The combination of user and project name is unique.

Authentication checks the X-API-Key header against the users table. If the key doesn't match any row, return a 401. No token expiration, no OAuth, no JWT.

Deploy to Cloud Run or Fly.io inside a Docker container. Connect to the managed PostgreSQL instance. Manually create a few user rows with API keys for the team to test with.

Deliverable: backend running in the cloud, accepting requests, returning mock responses, writing to the database.

Week 2: Wire real modules

Replace mock responses by integrating Person B's NLP parser and Person C's workflow engine into the interpret endpoint. When a request arrives: validate the API key, fetch the user's project state from the database, pass the user input plus project state to the NLP parser to get intent and entities, pass the intent, entities, and project state to the workflow engine to get an execution plan with generated Terraform files, and return everything to the CLI.

Build the state update logic for the state endpoint. When the CLI posts Terraform outputs after an apply, parse them and store them in the project state table. This is how the system remembers what infrastructure exists between requests.

Add state injection: before every NLP call, include the current project state in the context. The NLP model needs to know whether a cluster exists, what services are running, and what their configuration looks like.

Deliverable: the interpret endpoint returns real NLP results and real execution plans with real Terraform files. State persists between requests.

Week 3: Real execution support

Handle the multi-step nature of workflows. When the CLI reports a successful infrastructure setup, the state table should contain all the resource IDs needed for subsequent deploy operations (cluster ARN, subnet IDs, security group ID, ALB listener ARN, ECR URL).

Add error handling for common failure modes: OpenAI returning an unexpected response, missing project state fields, invalid entity values, database connection failures. Return clear error messages that the CLI can display without modification.

Test the full round trip: interpret a deploy request, verify the plan includes the right steps, verify the generated Terraform references the correct state values.

Deliverable: backend correctly supports the setup and deploy workflows end-to-end, with state updates persisted between operations.

Week 4: Remaining intents and fixes

Support the scale, stop, teardown service, and teardown infrastructure intents. These are simpler than deploy — scale and stop update the desired count in the service Terraform, teardown runs destroy.

Fix bugs discovered during Person D's integration testing. Handle edge cases: what if someone tries to deploy when no infrastructure exists, what if they try to teardown when nothing is running, what if the state is inconsistent with reality.

Deliverable: all 6 intents work through the backend. Edge cases handled gracefully.

Part 2 — Person B: NLP and Intelligence

Person B owns the OpenAI integration, intent definitions, prompt engineering, entity extraction, and accuracy.

Week 1: Intent system and parser

Define 6 intents as OpenAI function calling schemas. Each schema specifies the intent name, a description of when it applies, and the parameters it extracts.

The setup infrastructure intent extracts region (optional, defaults from project config) and optionally a cluster name and VPC CIDR. The deploy service intent extracts port, CPU, memory, replicas, environment variables, and service name — all optional since they have defaults from the project config. The scale service intent requires a replica count. The stop service intent has no required parameters. The teardown service intent optionally extracts a service name. The teardown infrastructure intent has no required parameters.

Write the system prompt that instructs the model. It should explain the supported intents, tell the model to fill missing values from project defaults, instruct it to use the project name as the service name when none is specified, and include rules like "if no infrastructure exists and the user wants to deploy, return setup_infra as the intent with a flag indicating deploy should follow." The prompt receives the current project state so the model can make context-aware decisions.

Build the parser module that calls OpenAI. It takes the user's text input and a context dictionary containing the project name, region, defaults, whether infrastructure exists, and what services are running. It populates the system prompt with this context, sends the request with the function schemas, and returns the parsed intent and entities.

Write 20 test cases and verify the parser handles them correctly. Include straightforward commands ("deploy my app on port 8080"), commands that need defaults ("deploy my app" — port should come from config), commands that need context ("scale to 5" — service name should come from state), ambiguous commands ("run it"), and nonsensical commands ("make me a sandwich" — should return an error, not an intent).

Deliverable: NLP parser works standalone with 90%+ accuracy on test cases. Ready to plug into Person A's backend.

Week 2: Live integration and context tuning

Person A wires the parser into the backend. Switch from standalone testing to real requests through the API.

Tune the system prompt based on integration results. Common issues to watch for: the model inventing entity values that weren't in the input, the model ignoring project defaults and asking for values it shouldn't need, the model returning setup_infra when infrastructure already exists (it should check the state context), and the model failing to extract environment variables from natural language ("deploy with NODE_ENV set to production").

Add entity validation: if the model returns a CPU value of 100 (which Fargate doesn't support), map it to the nearest valid value (256). Valid Fargate CPU/memory combinations are constrained — the validator should know them.

Test with Person D's CLI to ensure the full round trip works: natural language in, correct intent and entities out.

Deliverable: NLP works through the real backend with project state context. Common edge cases handled.

Week 3: Accuracy tuning

Focus on the deploy and setup intents since Person D is testing real AWS execution this week. Every NLP misparse that causes a wrong Terraform plan needs a prompt fix.

Tune the multi-step detection: if someone says "deploy my app" and state shows no infrastructure, the parser should return setup_infra with a flag indicating deploy_service should follow. The workflow engine uses this to build a combined plan.

Test with various phrasings: "launch my app", "get my application running", "put my app on AWS", "spin up my project". All should map to deploy_service. Similarly test scale variations: "scale to 5", "give me 5 replicas", "I need 5 instances", "bump it up to 5".

Deliverable: NLP accurately handles all variations of setup and deploy commands. Multi-step detection works.

Week 4: Remaining intents and hardening

Tune the scale, stop, and teardown intents. Scale should handle absolute values ("scale to 5") but not relative ("add 3 more" — that's cut from scope). Stop should be recognized from "stop my app", "shut it down", "turn it off". Teardown should distinguish between service-level ("remove my app") and infrastructure-level ("tear everything down", "destroy it all").

Fix any accuracy issues from week 3 testing. Add graceful error responses for inputs that don't match any intent: return a clear message suggesting what the user might have meant.

Deliverable: all 6 intents parse correctly across diverse phrasings.

Part 3 — Person C: Workflows and Code Generation

Person C owns the workflow engine, Terraform templates, the Jinja2 renderer, and serves as the AWS expertise on the team.

Week 1: Workflow engine and infrastructure template

Build the workflow engine. It takes an intent name, extracted entities, and project state, then returns an execution plan — an ordered list of steps. Each step has a name, a type (terraform apply, terraform destroy, or shell command), a description the CLI will display, and any generated files.

Define workflows for all 6 intents. The setup infrastructure workflow has one Terraform step that creates the VPC, ECS cluster, ALB, ECR repository, security groups, and IAM roles. The deploy service workflow has four steps: build Docker image (shell), authenticate with ECR (shell), push to ECR (shell), and apply the service Terraform (terraform). The scale service workflow has one Terraform step that re-renders the service template with a new desired count. The stop service workflow is the same as scale but with desired count set to zero. The teardown service workflow runs Terraform destroy on the service directory. The teardown infrastructure workflow runs Terraform destroy on the infrastructure directory after confirming no services are running.

The engine checks prerequisites before building a plan. If deploy is requested but no infrastructure exists, the engine should either return an error with a suggestion to set up infrastructure first, or prepend the setup steps if the NLP indicated a multi-step intent.

Write the infrastructure Terraform template. This is a single combined template (not split into separate files for speed of development) that creates: a VPC with public and private subnets across two availability zones, an internet gateway and NAT gateway, route tables, an ECS cluster with Fargate capacity provider and CloudWatch container insights, an Application Load Balancer with a default listener and security group, an ECR repository, a security group for ECS tasks that allows traffic only from the ALB, an IAM execution role for ECS tasks, and outputs for every resource ID the service template will need.

The template uses Jinja2 variables for project name, region, and VPC CIDR, with sensible defaults for all of them.

Build the Jinja2 renderer that takes a template name and a variables dictionary, renders the template, and returns file contents.

Deliverable: workflow engine returns plans for all 6 intents. Infrastructure template renders valid Terraform.

Week 2: Service template and backend integration

Write the ECS service Terraform template. This creates: a task definition (container image, port, CPU, memory, environment variables, CloudWatch log configuration), an ECS service (desired count, Fargate launch type, private subnet networking, ALB attachment), an ALB target group with health check, an ALB listener rule routing traffic to the service, and a CloudWatch log group.

This template takes variables from two sources. From the NLP entities: port, CPU, memory, replicas, environment variables, service name, image tag. From the project state: cluster ARN, VPC ID, private subnet IDs, ALB listener ARN, ECS task security group ID, ECR URL.

The renderer needs to merge both variable sources before rendering. Build this merging logic so the workflow engine passes a single context dictionary to the renderer.

Wire the workflow engine and codegen module into Person A's backend. The interpret endpoint should now call the workflow engine and return real plans with real Terraform file contents.

Test both templates: render with sample values, run terraform validate to confirm they produce valid Terraform. Test that the service template correctly references outputs from the infrastructure template (these come from project state, not from Terraform output references, since they're applied separately).

Deliverable: both Terraform templates render valid code. Workflow engine and codegen are integrated into the backend.

Week 3: Real AWS testing

Work with Person D to test against a real AWS account. Apply the infrastructure template and verify: VPC exists with correct subnets, ECS cluster is created, ALB is listening on port 80, ECR repository accepts pushes, security groups have the right rules.

Fix template bugs. Common issues: subnet CIDR conflicts, missing depends_on causing race conditions, IAM role not being ready when the ECS service tries to use it, ALB health check path not matching the application, security group rules too restrictive (tasks can't pull from ECR) or too permissive.

Test the deploy workflow end-to-end: build a test Docker image, push to ECR, apply the service Terraform, verify the ECS tasks start and become healthy behind the ALB.

Verify that Terraform outputs from both templates contain all the resource IDs needed and that Person A's backend stores them correctly.

Deliverable: infrastructure and service templates work on real AWS. Deploy produces a running, accessible application.

Week 4: Scale, stop, teardown and hardening

Test the scale workflow: re-render the service template with a different desired count, apply, verify the task count changes. Test scaling up and scaling down.

Test the stop workflow: set desired count to zero, verify tasks drain and stop while the service definition remains.

Test the teardown service workflow: Terraform destroy on the service directory, verify the service, task definition, target group, and listener rule are removed. Verify the infrastructure remains intact.

Test the teardown infrastructure workflow: destroy everything, verify VPC, cluster, ALB, ECR, and all associated resources are gone. This is the most dangerous operation — verify it doesn't leave orphaned resources that continue to incur charges.

Fix any template issues discovered during testing. Harden the templates: add lifecycle policies to prevent accidental deletion of critical resources, add tags to all resources for cost tracking, ensure the ECR repository has force_delete enabled so teardown doesn't fail on non-empty repositories.

Deliverable: all 6 workflows execute successfully against real AWS.

Part 4 — Person D: CLI and Execution

Person D owns the command-line interface, the local execution engine, the display layer, prerequisite checking, and integration testing. They're the consumer of everyone else's work and the first to discover integration problems.

Week 1: CLI skeleton

Build the CLI application with three commands. The init command creates an infrapilot.yaml project config file in the current directory with defaults for project name (inferred from directory name), region (us-east-1), CPU (256), memory (512), replicas (2), port (3000), and health check path (/health). The config command sets the backend API URL and API key, stored in a config file in the user's home directory. The chat command starts an interactive session.

Build the HTTP client module. It has two methods: interpret (sends user input, project name, and project config to the backend, returns the plan) and update_state (sends Terraform outputs to the backend after execution).

Build the display module using Rich. It shows the parsed intent and entities, renders the execution plan as a formatted table with step number, name, and type, asks "Proceed? y/N" for confirmation, shows a progress spinner for each step while it executes,

shows a checkmark or X when each step completes, and displays the final result (success with URL, or failure with error message).

Build the prerequisite checker. On startup, verify AWS credentials are configured by running `aws sts get-caller-identity`, verify Docker is installed and running by running `docker info`, and verify Terraform is installed by running `terraform version`. For each failure, display a clear message explaining what's missing and how to fix it.

Build the executor module. It wraps subprocess to run shell commands, streams stdout to the display line by line, and captures the exit code and stderr. It has specific methods for `terraform init`, `terraform plan`, `terraform apply` (which chains `init` → `plan` → `apply`), `terraform destroy`, and `terraform output` (which returns parsed JSON).

Deliverable: CLI runs interactively, sends to a backend (can use mock or real), displays formatted output, executes subprocess commands with streaming.

Week 2: Connect to real backend

Connect the CLI to Person A's deployed backend. The chat command now sends real requests and receives real plans with generated Terraform files.

Implement file handling: when the backend returns generated Terraform files, write them to a `.infrapilot/terraform/` directory inside the project. Separate infrastructure files from service files (different subdirectories) because they're applied and destroyed independently.

Implement the Terraform execution flow: write the generated files, run `terraform init` (only on first run or when files change), run `terraform plan` and display a summary of what will be created, run `terraform apply`, parse `terraform output` to extract resource IDs, and send those outputs to the backend's state endpoint.

Test the full flow with the infrastructure setup intent: user types "set up my infrastructure" → CLI sends to backend → backend returns plan with Terraform files → CLI displays plan → user confirms → CLI writes files and runs Terraform → infrastructure is created → CLI sends outputs to backend → backend updates state.

Deliverable: the setup infrastructure workflow works end-to-end through the real backend.

Week 3: Deploy flow and real AWS testing

Implement the Docker build and ECR push steps. The CLI receives these as shell commands in the execution plan. For the ECR authentication step, run `aws ecr get-login-password` and pipe it to `docker login`. For the build step, run `docker build` with the ECR URL as the tag. For the push step, run `docker push`.

Implement multi-step workflow execution. The plan contains multiple steps that execute in order. After each step, check the exit code. If it fails, display the error from stderr and stop. If it succeeds, move to the next step.

After the final Terraform apply step in a deploy workflow, send the Terraform outputs to the backend so the service is tracked in project state.

Work closely with Person C to test against real AWS. Deploy a test application (a simple Node.js or Python HTTP server) and verify it's accessible through the ALB URL.

Handle common error scenarios gracefully: Docker build fails because there's no Dockerfile (display "No Dockerfile found in current directory"), ECR push fails because credentials expired (auto-retry the login step), Terraform apply fails because a resource already exists (display the specific conflict).

Deliverable: full deploy workflow works — from "deploy my app" to a running container accessible via the ALB.

Week 4: Remaining intents and error hardening

Implement the scale, stop, and teardown workflows. For scale, the backend returns updated Terraform files with the new desired count — the CLI writes them and applies. For stop, same flow but desired count is zero. For teardown service, the CLI runs `terraform destroy` on the service directory. For teardown infrastructure, destroy the infrastructure directory but first check that no services are running.

Add a safety check for teardown infrastructure: list the services directory, warn the user if any service Terraform exists, and require explicit confirmation.

Test the complete lifecycle end-to-end: set up infrastructure → deploy → scale up → scale down → stop → start (re-deploy) → teardown service → teardown infrastructure. Every step should work smoothly.

Improve error messages based on testing. The most common failures should have specific, actionable messages rather than raw Terraform or Docker output.

Deliverable: all 6 operations work from natural language through the CLI. Error messages are clear.

Contracts to agree on before starting

These are the boundaries between each person's work. Agree on the exact formats before anyone writes their first line.

Between Person A and Person D: the request and response format for the interpret endpoint (what fields the CLI sends, what fields it receives back including the structure of steps and generated files) and the state endpoint (what Terraform outputs the CLI sends, how they're keyed).

Between Person B and Person C: the exact structure of intent plus entities. This is the output of NLP and the input to the workflow engine. It should include the intent name as a string, entities as a flat dictionary with known keys, and a multi-step flag indicating whether setup should precede deploy.

Between Person C and Person D: how generated Terraform files are structured in the response (dictionary of filename to content), which directory they go into locally, and how the service directory is named so teardown can find it.

Between all four: the error response format. Every error from the backend should have a consistent shape with an error flag, a machine-readable code, a human-readable message, and optionally a suggestion.